NLP Assignment 2 Report

Muhammad Umer Siddiqui
ms08702@st.habib.pk
Dhanani School of Science and Engineering, Habib University
Karachi, Pakistan

Abdullah Shaikh
as09245@st.habib.pk
Dhanani School of Science and Engineering, Habib University
Karachi, Pakistan

Arqish Zaria
az09714@st.habib.pk
Dhanani School of Science and Engineering, Habib University
Karachi, Pakistan

Abstract

This report covers language modeling and sentiment classification tasks. For Question 1, we implemented an N-Gram language model from scratch for unigram, bigram and trigram models using Add-k ($k = 0.1$) achieving the lowest perplexity of 16.8981 with the bigram model. For Question 2, we implemented a Naive Bayes classifier from scratch on a subset of the IMDB movie review dataset, reaching an F1-score of 0.816, and then improved it for the competition with negation handling, TF-IDF n-gram features, chi-square feature selection and Complement Naive Bayes, reaching a macro F1-score of 0.8861. For Question 3, we implemented a feedforward Artificial Neural Network from scratch with NumPy on a synthetic dataset of anonymized tokens.

Keywords

Natural Language Processing, N-Gram Language Model, Perplexity, Add-k Smoothing, Naive Bayes, Laplace Smoothing, TF-IDF, Chi-Square Feature Selection, Artificial Neural Network, Sentiment Classification, Backpropagation, One-Hot Encoding, ReLU, Sigmoid, Binary Cross Entropy

1 Question 1: Implementing an N-Gram Language Model and Testing Perplexity

1.1 Approach

1.1.1 Dataset Preparation. We loaded the training corpus (`train.txt`) and test corpus (`test.txt`). Both files were preprocessed by converting all text to lowercase, removing special characters and punctuation using regular expressions, and tokenizing the text into individual words by splitting on spaces. The training set has $N = 60$ total tokens and forms a vocabulary of $|V| = 30$ unique words. The test set has 31 tokens, out of which 7 words are OOV; `friends`, `plays`, `with`, `be`, `tricky`, `to`, `train`.

1.1.2 N-Gram Model and Smoothing. We implemented the N-gram language model using Python's `Counter` to count the frequencies of n-grams and $(n-1)$-gram contexts from the training data. Without smoothing, any unseen word or unseen n-gram in the test set gets a probability of zero which causes the perplexity to become undefined.

To handle unseen words, we tested both standard Add-One (Laplace) smoothing ($k = 1.0$) and Add-k smoothing ($k = 0.1$). Based on the results we chose Add-k ($k = 0.1$) as our main method. Add-k smoothing computes the probability of a word w_n given the previous $n-1$ words using:

$$P(w_1) = \frac{\text{count}(w_1) + k}{N + k \cdot |V|} \quad \text{(Unigram, } n = 1\text{)} \tag{1}$$

$$P(w_n \mid w_1^{n-1}) = \frac{\text{count}(w_1^n) + k}{\text{count}(w_1^{n-1}) + k \cdot |V|} \quad \text{(for } n > 1\text{)} \tag{2}$$

where $N = 60$ is the total number of training tokens, $|V| = 30$ is the vocabulary size and $k = 0.1$.

1.1.3 Why We Chose Add-k Instead of Add-One. Although Add-One ($k = 1.0$) avoids zero probabilities, it adds a +1 count for every word in the vocabulary. Since our training set is very small (only 60 tokens), most bigrams and trigrams occur only 1 or 2 times in training. Adding 30 to the denominator takes away too much probability mass from real phrases seen in training and gives it to unseen words. By using Add-k with $k = 0.1$, we add only $0.1 \times 30 = 3$ to the denominator. This still gives a non-zero probability to the 7 unseen test words while keeping much better probability estimates for the phrases that actually appear in `train.txt`.

1.2 Results

We evaluated the model on `test.txt` for $n = 1$ (Unigram), $n = 2$ (Bigram), and $n = 3$ (Trigram) using both our main Add-k ($k = 0.1$) smoothing and the baseline Add-One ($k = 1.0$) smoothing (Table 1).

Table 1: Q1: Perplexity Results on Test Set

Model	Add-k (0.1)	Add-One (1.0)
Unigram ($n = 1$)	44.8265	30.4586
Bigram ($n = 2$)	**16.8981**	**22.4829**
Trigram ($n = 3$)	23.0527	26.8136

1.3 Discussion: Impact of n on Performance and Perplexity

- **Unigram ($n = 1$):** The unigram model performs the worst because it ignores word order and context completely, predicting each word only from its frequency in training. With Add-One ($k = 1.0$), its perplexity is 30.4586 and with Add-k ($k = 0.1$) perplexity is 44.8265. With Add-k perplexity is higher because the 7 unseen test words get a much smaller probability ($\frac{0.1}{60+3} \approx 0.0016$ compared to $\frac{1}{60+30} \approx 0.0111$), and there is no context to boost known phrases.
- **Bigram ($n = 2$):** The bigram model gives the **best performance** using both methods, achieving a perplexity of 16.8981 with Add-k ($k = 0.1$) and 22.4829 with Add-One ($k = 1.0$). Conditioning on one previous word captures common two-word phrases shared between the training and test sets (such

as "the fox", "the dog", "the quick", "the lazy", "lazy dog", "dogs are" and "are loyal") without suffering from severe data sparsity.

- **Trigram ($n = 3$):** When n is increased to 3, perplexity rises back up to 23.0527 with Add-k ($k = 0.1$) and 26.8136 with Add-One ($k = 1.0$). Even though a trigram uses two previous words as context, our 60 word training set is too small to contain enough three word combinations. Very few trigrams from training repeat in the test set (mainly "the lazy dog" and "a lazy dog") so most test trigrams have a count of zero. This shows the key trade off of N-gram models is larger n captures more context but on a small dataset it suffers from data sparsity.

2 Question 2: Naive Bayes Classifier for Sentiment Analysis

2.1 Part 1: Naive Bayes from Scratch

2.1.1 Dataset Preparation. We used the IMDB Large Movie Review dataset [2]. With a fixed random seed (42), we randomly selected 500 positive and 500 negative reviews from the training set, and 100 positive and 100 negative reviews from the test set. Each review was converted to lowercase, punctuation was removed, and English stop words (NLTK list) were filtered out. The cleaned text was then tokenized into words with NLTK's `word_tokenize`. The vocabulary was built from the unique words of the 1,000 training reviews only, and each review is represented by the frequency of each vocabulary word it contains.

2.1.2 Model. The classifier was implemented without any machine learning library. The prior of each class c is the fraction of training reviews in that class:

$$P(c) = \frac{N_c}{N} \tag{3}$$

The likelihood of each vocabulary word w given a class uses Laplace (add-one) smoothing, where $\text{count}(w, c)$ is the number of times w appears in training reviews of class c and $|V|$ is the vocabulary size:

$$P(w \mid c) = \frac{\text{count}(w, c) + 1}{\sum_{w' \in V} \text{count}(w', c) + |V|} \tag{4}$$

A test review d is assigned the class with the highest posterior probability, computed in log space. Words not in the vocabulary are skipped:

$$\hat{c} = \arg\max_c \left[\log P(c) + \sum_{w \in d} \text{count}(w, d) \cdot \log P(w \mid c) \right] \tag{5}$$

2.1.3 Results. The classifier was evaluated on the 200 test reviews. It achieved an accuracy of 82.0% and an **F1-score of 0.816** for the positive class (Table 2).

Table 2: Q2 Part 1: Evaluation Results on Test Set

Metric	Value
Accuracy	82.00%
Precision (Positive)	0.8333
Recall (Positive)	0.8000
F1 Score (Positive)	0.8163
Precision (Negative)	0.8077
Recall (Negative)	0.8400
F1 Score (Negative)	0.8235
Macro F1	0.8199

Table 3: Q2 Part 1: Confusion Matrix

	Predicted Positive	Predicted Negative
Actual Positive	80	20
Actual Negative	16	84

The confusion matrix (Table 3) shows that the classifier performs consistently across both classes. With a precision of 0.833 and recall of 0.800 for the positive class, the model is slightly more conservative in predicting positive reviews: it is usually right when it does, but misses a small fraction of actual positive reviews. The errors are well balanced (20 false negatives against 16 false positives), so the model did not develop a strong bias toward either class despite being trained on only 1,000 reviews.

2.1.4 Implementation Challenges.

- **Floating-Point Underflow:** Multiplying many small probabilities for long reviews causes numerical underflow, producing a probability of zero. We avoided this by working in log space with `math.log()`, summing the log-probabilities of tokens instead of multiplying raw probabilities.
- **Zero-Frequency Problem:** A word that never appears in one class during training would get a zero likelihood and cancel the entire review's score for that class. This was handled with Laplace smoothing (adding 1 to every word count).

2.1.5 Ideas for Improvement.

- **N-grams:** Bigrams or trigrams would let the model capture local context and modifiers, especially negations (distinguishing "good" from "not good").
- **Better Preprocessing:** Stemming or lemmatization would group variants of a word ("running", "runs", "ran" to "run"), shrinking the vocabulary and strengthening word statistics. Removing HTML tags such as `<br />` would also clean the reviews.
- **TF-IDF Weighting:** Replacing raw counts with TF-IDF would down-weight very common words and let the model focus on discriminative vocabulary.
- **More Training Data:** The model was trained on 1,000 of the 25,000 available training reviews. Using more data would improve vocabulary coverage and generalization.

2.2 Part 2: Naive Bayes Performance Competition

2.2.1 Approach. Our competition model follows the pipeline of Hamzah [1], who classified IMDB reviews by combining TF-IDF features, chi-square feature selection and Multinomial Naive Bayes, and showed that chi-square selection raised accuracy from 81.39% to 85.37%. We kept this TF-IDF and chi-square pipeline, used Complement Naive Bayes in place of Multinomial Naive Bayes, and did not use the AdaBoost step. Part 2 uses scikit-learn, while the Part 1 code is unchanged. Compared to the Part 1 baseline, we changed the following:

- **Training data:** all 25,000 training reviews (12,500 per class) instead of 1,000.
- **Text cleaning:** HTML tags are removed, text is lowercased and punctuation is removed. Stop words are no longer removed.
- **Negation handling:** after a negation word (e.g. "not", "no", "never" or a word ending in "n't"), every following word is prefixed with `NOT_` until the next punctuation mark, so "not good" produces the feature `NOT_good`.
- **Features:** TF-IDF over unigrams, bigrams and trigrams with sublinear term frequency, ignoring n-grams that appear in fewer than 3 reviews or in more than 95% of reviews. This gives 488,632 features.
- **Feature selection:** the top k features by chi-square score [1].
- **Classifier:** Complement Naive Bayes with smoothing parameter α.

2.2.2 Tuning. The number of selected features $k \in \{40000, 60000, 80000, 100000\}$ and the smoothing parameter $\alpha \in \{0.05, 0.1, 0.2, 0.3, 0.5\}$ were tuned with 5-fold cross-validation on the training set, using macro F1 as the score. The test set was not used for tuning. The best configuration was $k = 40000$ and $\alpha = 0.5$, with a cross-validation macro F1 of 0.8710.

2.2.3 Results. The final model was trained on all 25,000 training reviews and evaluated on all 25,000 test reviews (Tables 4 and 5). It reached a **macro F1-score of 0.8861**, compared to 0.8199 for the Part 1 baseline. The result can be reproduced by running `Q2/part2.py` with the `aclImdb` folder placed inside `Q2`.

Table 4: Q2 Part 2: Evaluation Results on Test Set

Metric	Value
Accuracy	88.61%
Precision (Positive)	0.8906
Recall (Positive)	0.8802
F1 Score (Positive)	0.8854
Precision (Negative)	0.8816
Recall (Negative)	0.8919
F1 Score (Negative)	0.8867
Macro Precision	0.8861
Macro Recall	0.8861
Macro F1	0.8861

Table 5: Q2 Part 2: Confusion Matrix

	Predicted Positive	Predicted Negative
Actual Positive	11003	1497
Actual Negative	1351	11149

Compared to Part 1, the errors stay balanced between the classes (1,497 false negatives against 1,351 false positives) while the error rate drops from 18.0% to 11.4%. In cross-validation, the smallest feature set ($k = 40000$) scored best for every value of α, and keeping more features generally lowered the score. This agrees with Hamzah [1], where a small chi-square selection (2,000 features) gave the best accuracy: many of the 488,632 n-grams add noise rather than sentiment information. A wider search toward smaller k and larger α, where the best values lay at the edge of our grid, may improve the result further.

3 Question 3: Artificial Neural Network for Sentiment Classification

3.1 Approach

3.1.1 Dataset Preparation. We loaded the synthetic dataset which contains 5000 training and 500 test samples. Each sample has a sentence made up of anonymized tokens like `word_1`, `word_2` and a binary label (0 or 1). We cleaned the text by converting to lowercase and removing punctuation, then tokenized each sentence by splitting on spaces. A vocabulary of 300 unique words was built from the training set only. Each sentence was then converted into a one-hot encoded vector of size 300.

3.1.2 Network Architecture. We implemented a simple feedforward ANN with one hidden layer from scratch using only NumPy:

- Input layer: 300 neurons (vocabulary size)
- Hidden layer: 128 neurons with ReLU activation
- Output layer: 1 neuron with Sigmoid activation

3.1.3 Forward Propagation. The forward pass computes:

$$Z_1 = X \cdot W_1 + b_1 \tag{6}$$

$$A_1 = \text{ReLU}(Z_1) \tag{7}$$

$$Z_2 = A_1 \cdot W_2 + b_2 \tag{8}$$

$$A_2 = \text{Sigmoid}(Z_2) \tag{9}$$

3.1.4 Loss Function. We used Binary Cross Entropy loss:

$$L = -\frac{1}{n} \sum [y \cdot \log(A_2) + (1 - y) \cdot \log(1 - A_2)] \tag{10}$$

3.1.5 Backpropagation. We computed gradients layer by layer using the chain rule.

Output Layer Gradients. Starting from the loss, we differentiate with respect to A_2:

$$\frac{\partial L}{\partial A_2} = -\frac{y}{A_2} + \frac{1 - y}{1 - A_2} \tag{11}$$

The derivative of Sigmoid is:

$$\frac{\partial A_2}{\partial Z_2} = A_2(1 - A_2) \tag{12}$$

Applying the chain rule:

$$\delta Z_2 = \frac{\partial L}{\partial A_2} \cdot \frac{\partial A_2}{\partial Z_2} = \left(-\frac{y}{A_2} + \frac{1-y}{1-A_2}\right) \cdot A_2(1-A_2) = A_2 - y \quad (13)$$

Since $Z_2 = A_1 \cdot W_2 + b_2$:

$$\delta W_2 = \frac{1}{n} A_1^T \cdot \delta Z_2 \quad (14)$$

$$\delta b_2 = \frac{1}{n} \sum \delta Z_2 \quad (15)$$

Hidden Layer Gradients. Propagating the error back through W_2:

$$\delta A_1 = \delta Z_2 \cdot W_2^T \quad (16)$$

The derivative of ReLU is:

$$\text{ReLU}'(Z_1) = \begin{cases} 1 & \text{if } Z_1 > 0 \\ 0 & \text{if } Z_1 \leq 0 \end{cases} \quad (17)$$

Applying the chain rule:

$$\delta Z_1 = \delta A_1 \times \text{ReLU}'(Z_1) \quad (18)$$

Since $Z_1 = X \cdot W_1 + b_1$:

$$\delta W_1 = \frac{1}{n} X^T \cdot \delta Z_1 \quad (19)$$

$$\delta b_1 = \frac{1}{n} \sum \delta Z_1 \quad (20)$$

Weight Updates. Weights were updated using gradient descent:

$$W = W - \alpha \cdot \delta W \quad (21)$$

$$b = b - \alpha \cdot \delta b \quad (22)$$

where $\alpha = 0.1$ is the learning rate.

3.2 Results

Table 6: Evaluation Results on Test Set

Metric	Value
Accuracy	52.60%
Precision (Positive)	0.5149
Recall (Positive)	0.9641
F1 Score (Positive)	0.6713
Precision (Negative)	0.7000
Recall (Negative)	0.0843
F1 Score (Negative)	0.1505
Macro F1	0.4109

Table 7: Confusion Matrix

	Predicted 0	Predicted 1
Actual 0	21	228
Actual 1	9	242

3.2.1 Confusion Matrix. The model predicts almost everything as class 1, as shown by the high recall for the positive class (0.9641) and near-zero recall for the negative class (0.0843). This imbalance is reflected in the large off-diagonal entry of 228 false positives against only 21 true negatives.

3.3 Challenges

The main challenge was the nature of the dataset. All 300 words of the vocabulary appear in both positive and negative sentences. This means there is no meaningful signal for the model to learn from, the words do not distinguish between the two classes. Secondly, one-hot encoding produces sparse vectors where around 97% of values are zeros, and the model ends up predicting randomly. The loss stayed at 0.6931 throughout training.

3.4 Potential Improvements

- Using TF-IDF instead of one-hot encoding to give words different weights based on their discriminative power
- Adding more hidden layers to increase model capacity
- Using a real labeled sentiment dataset instead of synthetic anonymized tokens, where word choice actually correlates with the label.

3.5 Conclusion of Question 3

This part implemented a feedforward ANN from scratch using NumPy for binary sentiment classification. The network was trained correctly and the backpropagation derivations were verified, but the model could not learn because the synthetic dataset provides no discriminative signal: all vocabulary words appear with similar frequencies in both classes. The result is a model that collapses to predicting the majority class, achieving 52.60% accuracy with a loss fixed at 0.6931 throughout training.

4 Overall Conclusion

Across all three questions, we see that text representation, context length and data quality matter as much as the model itself. In Question 1, the bigram model with Add-k ($k = 0.1$) smoothing achieved the lowest perplexity (16.8981) by capturing local two word context without suffering from the severe data sparsity of trigrams or the heavy penalty of Add One smoothing. On real IMDB reviews in Question 2, a from-scratch Naive Bayes classifier reached an F1-score of 0.816, and better preprocessing, negation handling, TF-IDF n-gram features and chi-square feature selection raised the macro F1-score to 0.8861 while keeping a Naive Bayes model. Finally, on the synthetic dataset of Question 3, where every word appears in both classes, even a correctly implemented neural network could not learn because the input features carried no information about the label.

References

[1] Muhamad Biki Hamzah. 2021. Classification of Movie Review Sentiment Analysis Using Chi-Square and Multinomial Naïve Bayes with Adaptive Boosting. *Journal of Advances in Information Systems and Technology* 3, 1 (April 2021), 67–74. https://journal.unnes.ac.id/sju/jaist/article/view/49098/19796

[2] Andrew L. Maas, Raymond E. Daly, Peter T. Pham, Dan Huang, Andrew Y. Ng, and Christopher Potts. 2011. Learning Word Vectors for Sentiment Analysis. In *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*. 142–150.